

CESAR Lucas

BIGET Robin

Rapport du projet World of IUT

Introduction

Dans le cadre de la SAÉ R1.01 "Initiation au développement", nous avons réalisé en binôme une version basique d'un jeu d'aventure textuelle intitulé "World of IUT". Ce projet avait pour objectif principal la création d'un système d'exploration textuelle avec des améliorations optionnelles telles que la gestion d'objets ou le chargement d'univers depuis un fichier.

Ce rapport présente les choix de structures de données, les fonctionnalités implémentées, les limitations identifiées (améliorations possibles) et les tests réalisés, ainsi que la répartition du travail.

Choix des structures de données

Structures principales :

1. Structure Game :

- **Description** : La structure centrale du jeu qui regroupe tous les éléments essentiels, notamment le joueur, les localisations et la pile de gestion mémoire.
- **Champs** :
 - Mobile *player : Pointeur vers la structure représentant le joueur.
 - Stack *locations : Pile contenant toutes les localisations du jeu.
- **Raison du choix** : Cette centralisation facilite la gestion globale des éléments du jeu et permet une manipulation simplifiée lors des différentes actions du joueur.

2. Structure Location :

- **Description** : Représente une case du monde, avec des informations sur son nom, sa description, et ses connexions vers d'autres cases.
- **Champs** :
 - char *name : Nom de la case.
 - char *desc : Description longue de la case.
 - Location *directions[DIRECTIONS] : Tableau de pointeurs vers les localisations adjacentes (Nord, Est, Sud, Ouest, Haut, Bas).
- **Raison du choix** : Le tableau de pointeurs permet une navigation rapide et intuitive entre les cases, tout en étant flexible pour étendre le monde.

3. Structure Mobile :

- **Description** : Représente le joueur, avec ses caractéristiques et sa localisation actuelle.
- **Champs** :
 - char *name : Nom du joueur.
 - char *desc : Description du joueur.
 - Location *currentLocation : Pointeur vers la case où se trouve actuellement le joueur.
- **Raison du choix** : Permet une représentation claire du joueur et de son interaction avec l'environnement du jeu.

4. Structure StackCell :

- **Description** : Représente une cellule de la pile.
- **Champs** :
 - Location *location : Pointeur vers une location.
 - Struct StackCell *next : Pointeur vers la cellule suivante.
 - Typedef StackCell *Stack : Définition du type Stack comme un pointeur vers la première cellule de la pile.
- **Raison du choix** : Permet de libérer toutes les locations une seule fois sans avoir à se soucier des locations voisines.

Les structures sont conçues pour travailler ensemble :

- La pile Stack dans Game garantit que toutes les localisations créées sont tracées et peuvent être correctement détruites.
- La structure Game agit comme un conteneur principal, reliant les différents modules de manière logique et ordonnée.

Fonctionnalités et Problèmes

Fonctionnalités implémentées

Les fonctionnalités implémentées permettent plusieurs actions essentielles. Tout d'abord, les déplacements dans le monde sont possibles grâce aux commandes go <direction>, qui permettent de naviguer entre les cases connectées. Ensuite, l'exploration est facilitée par la commande look <direction>, qui affiche des descriptions détaillées de la localisation courante ou des directions adjacentes. Enfin, la gestion de la mémoire est assurée grâce à l'utilisation de la structure Stack, permettant de libérer proprement les localisations. Ces fonctionnalités offrent une navigation fluide, une exploration précise et une gestion efficace des ressources.

Tests réalisés

Les scénarios de tests incluent plusieurs aspects essentiels. Tout d'abord, il est nécessaire de vérifier la navigation simple en testant les déplacements entre les cases d'un monde minimal, comme indiqué dans le schéma du monde. Par exemple, on peut utiliser la commande go nord depuis la case 5 pour atteindre la case 6. Ensuite, l'exploration doit être validée en s'assurant que la commande look permet bien d'afficher les descriptions des cases. Enfin, il est important de gérer les erreurs en testant tous les cas d'utilisation incorrects, comme tenter un déplacement avec la commande go haut lorsqu'aucune issue n'existe dans cette direction.

Tous les scénarios de test ont été réussis pour les fonctionnalités implémentées. Chaque test réalisé a été fait au fur et à mesure de la programmation du jeu.

Répartition du travail :

Tâche	Membres
Première étape : comprendre le matériau fourni	CESAR Lucas, BIGET Robin
Deuxième étape : le module Location	CESAR Lucas
Troisième étape : créer et connecter les Location	CESAR Lucas
Quatrième étape : une gestion correcte de la mémoire	BIGET Robin

Nous avons décidé de nous partager la charge de travail en fonction des différentes étapes à réaliser (4 étapes). Cela nous permet de bien comprendre l'étape qui nous est attribuée et de la faire de A à Z.

Problèmes principaux rencontrés :

Problèmes liés à la commande go :

Une fois fini de configurer la commande go, lorsque j'ai commencé à tester cette commande, le programme se terminait peu importe la localisation que je donnais. En déboguant le programme j'ai remarqué que peu importe la direction que je donnais, cela me renvoyait toujours -1. Mais -1 correspond à WRONGDIR, cela indique que ce n'est pas la bonne direction est renvoie un message d'erreur. Finalement j'ai remarqué que ma conversion n'était pas bonne car elle me renvoyait toujours -1 alors j'ai vérifié ma fonction LocationSetExit, puis j'ai refait la conversion correctement.

Ensuite j'ai rencontré un autre problème, lorsque je testais ma commande go, la conversion cette fois-ci fonctionnait correctement mais le joueur ne se déplaçait pas dans la localisation suivante. Cela venait de la fonction CmdGo, lorsque je faisais mes différents tests pour pouvoir déplacer le joueur dans la prochaine localisation, j'utilisais que des « if ». Or, quand on fait ce genre d'opération, c'est pour tester chaque condition indépendamment, mais ce n'est pas ce qui était attendu. Car pour déplacer le joueur, ce que l'on veut, c'est que dès

qu'une condition est vraie, le reste des conditions est ignoré. Alors il fallait utiliser des « else if », cela améliore l'efficacité et garantit qu'une seule branche est exécutée.

Problème structure Stack :

Lors de l'ajout d'un champ de type Stack dans la structure Game j'ai fait l'erreur d'employé deux fois le même nom « location » dans la structure alors que cela n'est pas possible. Afin de pallier à cette erreur j'ai dû modifier le nom « location » par « locationlist ».

Plusieurs problèmes sont apparus à cause de la mal compréhension de l'utilisation des différents types, notamment entre « int » et « stack ».

Plusieurs messages d'erreur sont apparus concernant des fichiers .h non inclus.

```
make: *** [<built-in>: WorldOfIUT.o] Error 1
PS C:\Users\Perso\Documents\Mes_cours\Semestre_1\Gitlab\SAE 1.02\WorldOfIUT\world-of-iut> make
cc -c -o WorldOfIUT.o WorldOfIUT.c
In file included from location.h:6,
                 from mobile.h:3,
                 from game.h:4,
                 from WorldOfIUT.c:2:
stack.h:14:5: error: unknown type name 'Location'
 14 |     Location *location;           // Pointeur vers une Location
    |     ~~~~~
stack.h:30:30: error: unknown type name 'Location'
 30 | Stack StackPush(Stack stack, Location *location);
    |                               ~~~~~
stack.h:33:1: error: unknown type name 'Location'
 33 | Location *StackHead(Stack stack);
    | ~~~~~
make: *** [<built-in>: WorldOfIUT.o] Error 1
PS C:\Users\Perso\Documents\Mes_cours\Semestre_1\Gitlab\SAE 1.02\WorldOfIUT\world-of-iut> █
```

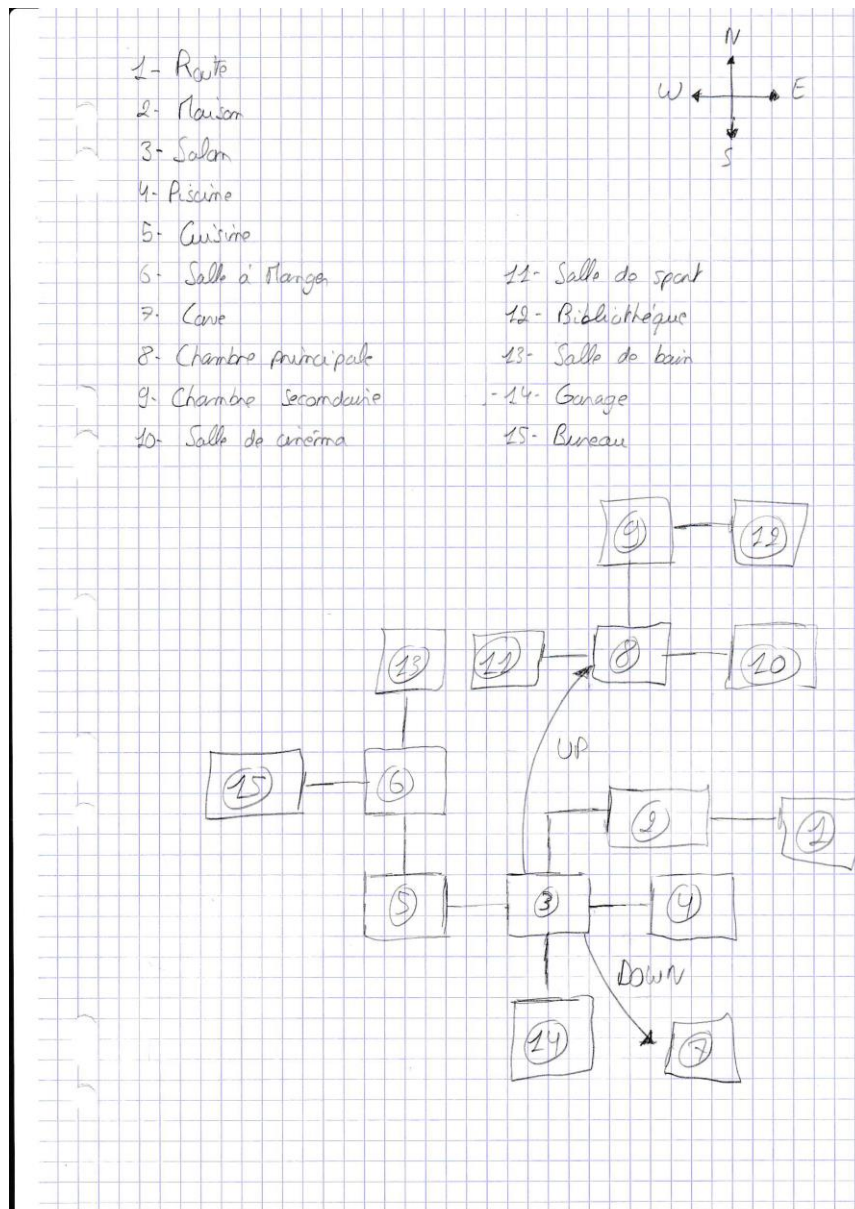
Pourtant tous les fichiers .h étaient bien inclus ! Le problème était qu'il y avait une dépendance circulaire car si location.h inclut stack.h et stack.h inclut location.h, cela crée une boucle où aucun des deux fichiers ne peut "terminer" l'autre. Cela empêche le compilateur de comprendre complètement les types.

Pour résoudre ce problème il fallait appliquer une déclaration anticipée c'est-à-dire ajouter ceci dans stack.h :

```
struct Location;
```

```
typedef struct Location Location;
```

Plan du jeu + explication :



Voici le plan de notre jeu textuel, ce plan représente tous les chemins possibles que l'on peut empruntés en utilisant les différentes directions (nord, est, sud, ouest, haut et bas).

Notre jeu consiste à pouvoir se déplacer dans une maison afin de découvrir toutes les pièces qui s'y trouve avec leurs décorations. Lorsque l'on commence ce jeu nous somme à ma case n°1 qui la route, donc le joueur se trouve sur une route et il peut que se déplacer vers la maison pour la découvrir.

Conclusion

Ce projet nous a permis de développer nos compétences en C, notamment au niveau de l'utilisation et de la compréhension des pointeurs et de la gestion mémoire. Bien que certaines améliorations soient possibles, les objectifs principaux ont été atteints.

Améliorations possibles :

Pour que notre jeu textuel soit vraiment complet, il aurait fallu que l'on crée, des objets par exemple, et que l'on puisse interagir avec ceux-ci. Ou même créer encore plus de localisations permettant de créer un petit village, par exemple et de pouvoir le découvrir entièrement, cela donnerait plus de sens au jeu et permettrait de l'enrichir.